

Air University Islamabad Department of Computer Science



LAB-2

SUBMITTED 20 Sep, 2026

DSA LAB Mam. Shamsa Kanwal

Submitted By:

Imad Khan	2500623
-----------	---------

Lab Objectives:

In this lab, we learned the

- Understand Doubly Linked List.
- Understand Circular Linked List.
- Implement insertion and deletion.
- Perform forward and backward traversal.
- Implement list reversal.
- Apply Circular Linked List in Josephus problem.

Task 1 — Circular Linked List: Insertion & Traversal

Implement a Circular Linked List class with `insertAtEnd(int value)` and `display()`. The `display()` function must print each node once and stop after returning to the head. Print the list followed by (back to head).

CODE:

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
};

class CircularLinkedList {
private:
    Node* head;

public:
    CircularLinkedList() {
        head = nullptr;
    }

    void insertAtEnd(int value) {
        Node* newNode = new Node();
        newNode->data = value;

        if (head == nullptr) {
            head = newNode;
            newNode->next = head;
            return;
        }

        Node* temp = head;

        while (temp->next != head)
            temp = temp->next;

        temp->next = newNode;
        newNode->next = head;
    }

    void display() {
        if (head == nullptr) {
            cout << "List is empty" << endl;
            return;
        }

        Node* temp = head;

        do {
            cout << temp->data << " -> ";
            temp = temp->next;
        } while (temp != head);

        cout << "(back to head)" << endl;
    }
}
```

```
};

int main() {
    CircularLinkedList list;

    list.insertAtEnd(10);
    list.insertAtEnd(20);
    list.insertAtEnd(30);
    list.insertAtEnd(40);

    list.display();

    return 0;
}
```

Output



```
10 -> 20 -> 30 -> 40 -> (back to head)
```

Task 2 — Circular Linked List: Deletion

Add a `deleteValue(int value)` function to your Circular Linked List from Task 1. Handle deleting the only node, the head node, and any other node. Ensure the last node's `next` points correctly to the new head.

CODE:

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
};

class CircularLinkedList {
private:
    Node* head;

public:
    CircularLinkedList() {
        head = nullptr;
    }

    void insertAtEnd(int value) {
        Node* newNode = new Node();
        newNode->data = value;

        if (head == nullptr) {
            head = newNode;
            newNode->next = head;
            return;
        }

        Node* temp = head;
```

```

    while (temp->next != head)
        temp = temp->next;

    temp->next = newNode;
    newNode->next = head;
}

void deleteValue(int value) {
    if (head == nullptr)
        return;

    Node* current = head;
    Node* previous = nullptr;

    // Only one node
    if (head->next == head) {
        if (head->data == value) {
            delete head;
            head = nullptr;
        }
        return;
    }

    // Delete head
    if (head->data == value) {
        Node* last = head;

        while (last->next != head)
            last = last->next;

        head = head->next;
        last->next = head;

        delete current;
        return;
    }

    // Delete other node
    previous = head;
    current = head->next;

    while (current != head && current->data != value) {
        previous = current;
        current = current->next;
    }

    if (current != head) {
        previous->next = current->next;
        delete current;
    }
}

void display() {
    if (head == nullptr) {
        cout << "List is empty" << endl;
        return;
    }
}

```

```

    Node* temp = head;

    do {
        cout << temp->data << " -> ";
        temp = temp->next;
    } while (temp != head);

    cout << "(back to head)" << endl;
}
};

int main() {
    CircularLinkedList list;

    list.insertAtEnd(10);
    list.insertAtEnd(20);
    list.insertAtEnd(30);
    list.insertAtEnd(40);

    cout << "Original List: ";
    list.display();

    list.deleteValue(10);

    cout << "After deleting 10: ";
    list.display();

    list.deleteValue(30);

    cout << "After deleting 30: ";
    list.display();

    return 0;
}

```

Output

```

Original List: 10 -> 20 -> 30 -> 40 -> (back to head)
After deleting 10: 20 -> 30 -> 40 -> (back to head)
After deleting 30: 20 -> 40 -> (back to head)

```

Task 3 — Reverse a Doubly Linked List

Using the `DoublyLinkedList` class from Part C, write `reverseList()` that reverses the list in place by swapping each node's `prev` and `next` pointers. Update `head` to the old tail. Demonstrate the list before and after reversing.

CODE:

```

#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* prev;
    Node* next;
};

```

```

class DoublyLinkedList {
private:
    Node* head;

public:
    DoublyLinkedList() {
        head = nullptr;
    }

    void insertAtEnd(int value) {
        Node* newNode = new Node();
        newNode->data = value;
        newNode->next = nullptr;

        if (head == nullptr) {
            newNode->prev = nullptr;
            head = newNode;
            return;
        }

        Node* temp = head;

        while (temp->next != nullptr)
            temp = temp->next;

        temp->next = newNode;
        newNode->prev = temp;
    }

    void reverseList() {
        Node* current = head;
        Node* oldTail = nullptr;

        while (current != nullptr) {
            oldTail = current;

            Node* temp = current->prev;
            current->prev = current->next;
            current->next = temp;

            current = current->prev;
        }

        head = oldTail;
    }

    void displayForward() {
        Node* temp = head;

        while (temp != nullptr) {
            cout << temp->data << " <-> ";
            temp = temp->next;
        }

        cout << "NULL" << endl;
    }
};

int main() {

```

```

DoublyLinkedList list;

list.insertAtEnd(10);
list.insertAtEnd(20);
list.insertAtEnd(30);
list.insertAtEnd(40);

cout << "Before Reverse: ";
list.displayForward();

list.reverseList();

cout << "After Reverse: ";
list.displayForward();

return 0;
}

```

Output



```

Before Reverse: 10 <--> 20 <--> 30 <--> 40 <--> NULL
After Reverse: 40 <--> 30 <--> 20 <--> 10 <--> NULL

```

Task 4 — Josephus-Style Round Robin

Using your Circular Linked List, write a program that inserts **n players (1..n)** and repeatedly removes every **k-th player** until one remains. Print the elimination order and final winner. Use **n = 7, k = 3** as the test case.

CODE:

```

#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
};

class CircularLinkedList {
private:
    Node* head;

public:
    CircularLinkedList() {
        head = nullptr;
    }

    void insertAtEnd(int value) {
        Node* newNode = new Node();
        newNode->data = value;

        if (head == nullptr) {
            head = newNode;
            newNode->next = head;
            return;
        }
    }
}

```

```

    }

    Node* temp = head;

    while (temp->next != head)
        temp = temp->next;

    temp->next = newNode;
    newNode->next = head;
}

void josephus(int k) {
    Node* current = head;
    Node* previous = nullptr;

    cout << "Elimination Order: ";

    while (current->next != current) {

        for (int i = 1; i < k; i++) {
            previous = current;
            current = current->next;
        }

        cout << current->data << " ";

        previous->next = current->next;
        delete current;

        current = previous->next;
    }

    cout << endl;
    cout << "Winner: " << current->data << endl;

    delete current;
    head = nullptr;
}

};

int main() {
    CircularLinkedList list;

    int n = 7;
    int k = 3;

    for (int i = 1; i <= n; i++) {
        list.insertAtEnd(i);
    }

    list.josephus(k);

    return 0;
}

```

Output



Elimination Order: 3 6 2 7 5 1

Winner: 4

---END---